D02 Architecture and Data Schema

D02 — Architecture and data schema

Branch: `cursor/d02-backend-cicd-38d2`
Scope: Backend, REST API, database (incubation funding deliverable D02)
Production: unchanged by this work (**no production redeploy**)

Status legend used below:

- **Implemented & verified** — present in code and exercised by local and/or CI evidence on this branch
- **Present but unused / optional** — code exists; not part of the verified D02 path
- **Future / not configured** — documented option only; not claimed as completed for D02

1. Architecture (implemented & verified)

```
Mobile / Web client (Expo)
        │  HTTPS JSON  (Bearer JWT — app HS256)
        ▼
Vercel Serverless entry
  api/v1/[...path].ts
        │
        ▼
lib/server/vercel-dispatch.ts
  resolves /api/v1/* → app/api/v1/**/route.ts handlers
        │
        ├── AuthN verification (active):
        │     lib/server/auth-helpers.ts
        │     → lib/server/app-jwt.ts
        │     Authorization: Bearer <JWT>
        │     HS256, issuer "grpd", secret AUTH_JWT_SECRET
        │
        ├── AuthN issuance (active):
        │     POST /api/v1/auth/email/register
        │     POST /api/v1/auth/email/login
        │     POST /api/v1/auth/logout
        │     POST /api/v1/auth/pin/login  (fixture allowlist;
CI/local only when PIN_ALLOWLIST_JSON is set)
        │
```

```
        ├── AuthZ:
        │     lib/server/role-checks.ts
        │     approved membership + role → permission
        │
        └── Persistence:
              Prisma Client → PostgreSQL
              (project hosting target: Neon PostgreSQL; D02 evidence
used disposable local/CI Postgres)
              prisma/schema.prisma + prisma/migrations/*
```

Active authentication (implemented & verified)

1. Client obtains tokens from **email register/login** (scrypt password hash in `AppUser`, JWT signed by `signAppAccessToken`).
2. Protected handlers call `await requireAuth(req)`.
3. `getAuthUserId` accepts only `Authorization: Bearer …` and verifies the token with **`verifyAppAccessToken`** (`lib/server/app-jwt.ts`).
4. Missing/invalid Bearer → `401` with code `UNAUTHORIZED` (captured in evidence).

Database for the product is **Neon PostgreSQL** (via `DATABASE_URL` / Prisma). There is **no Supabase database** in this project's D02 setup.

Optional / unused Supabase JWT helper (present but unused)

`lib/server/supabase-jwt.ts` defines `verifySupabaseAccessToken`.
`getAuthUserId` still calls it as a **fallback after** app-JWT verification fails.

Status for D02:

- The helper only attempts verification when `SUPABASE_URL` is set (and optionally `SUPABASE_JWT_SECRET`).
- D02 local evidence and CI **do not set** those variables; tests explicitly clear them.
- No Supabase Auth product, JWKS endpoint, or Supabase-backed user store is part of the verified D02 flow.
- Therefore D02 **does not claim** Supabase authentication as an implemented capability. Treat the helper as leftover optional code, not as the active auth architecture.

Note: `app-jwt.ts` can also read `SUPABASE_JWT_SECRET` as an alternate env name for the **same app HS256 signing secret** if `AUTH_JWT_SECRET` is unset. That is an env alias for the app JWT secret, not evidence of Supabase Auth usage.

Request path (implemented & verified)

1. Client calls `/api/v1/...` with JSON when needed.
2. `vercel.json` rewrites `/api/v1/:path*` → `api/v1/[...path].ts`.
3. Dispatcher loads the matching handler under `app/api/v1/**`.
4. Protected handlers `await requireAuth(req)` → `401` when unauthenticated.
5. Governance handlers also `await requireClubPermission(...)` → `403` when forbidden.

Environments

Env	Status	Database	API surface	N
Local disposable (evidence)	**Verified**	Local PostgreSQL created by `scripts/d02-setup-local-db.sh`	In-process dispatcher via Jest / `d02-evidence-capture.ts`	Synthetic secre gitignored `.en`
CI	**Verified**	GitHub Actions Postgres 14 (`55439`)	Same handlers via Jest	`.github/work verificatior` this branch
Hosted Vercel Preview for this branch	**Future / not claimed as D02 verification**	Would need Preview-only `DATABASE_URL` + `AUTH_JWT_SECRET`	Preview URL after PR push	Not used as the functional proc
Production	**Unchanged**	Existing Neon / Vercel production	`grpd30.vercel.app`	**Not redeploye**

2. Data schema (Prisma) — implemented

Models (after migration `20261031020000_add_app_users`)

Model	Table	Purpose
`AppUser`	`app_users`	Email/password accounts (`email` unique, `passwordHash`, optional `displayName`)
`Club`	`clubs`	Club identity (`slug` unique), `visibility` (`public`|`members`), `createdById`
`ClubMembership`	`club_memberships`	Unique (`userId, clubId`); `role`, `status`, optional `displayName` / `prSummary`
`ClubInvite`	`club_invites`	Invite `code` unique; optional `expiresAt`
`Session`	`sessions`	Outing; optional `clubId`; `visibility`; JSON `paceGroups`; host fields
`SessionAttendance`	`session_attendance`	Unique (`sessionId, userId`); `status`; optional `groupId`
`SessionTag`	`session_tags`	Per-session tagged person

Enums (schema)

- `MembershipRole`: `member` | `coach` | `admin`

- `MembershipStatus`: `pending` | `approved` | `rejected` | `banned`
- `SessionVisibility` / `ClubVisibility`: `public` | `members`
- `AttendanceStatus`: `joined` | `suggested` | `left` | `requested` | `waitlisted` | `declined` (legacy)

Relationships

```
AppUser.id ── opaque string userId ──► ClubMembership /
SessionAttendance / Session.hostUserId
  (no Prisma FK from membership/attendance to AppUser)

Club 1──N ClubMembership
Club 1──N ClubInvite
Club 1──N Session   (Session.clubId nullable)
Session 1──N SessionAttendance
Session 1──N SessionTag
```

Migrations applied on the D02 test database (verified)

1. `20250128000000_add_clubs_memberships`
2. `20250128100000_add_session_tags`
3. `20260130090000_add_membership_prs`
4. `20260130235238_add_session_fields`
5. `20260210022611_add_attendance_status_suggested_left`
6. `20261031020000_add_app_users` ← added for email auth

Transcript: `evidence/migrations.txt`.

3. Authorization rules (implemented)

From `lib/server/role-checks.ts` (approved membership required):

Permission	Who
`view`, `join`	any **approved** member
`create_session`	approved `member`, `coach`, or `admin`
`approve_members`, `invite`, `manage_club`	approved `coach` or `admin`

Route mapping verified in handlers:

Endpoint	Auth	Role check
`GET /api/v1/clubs/:id/roster`	required	`manage_club` → else `403` (**captured**)
`POST /api/v1/clubs/:id/invite`	required	`invite`
`POST /api/v1/clubs/:id/approve`	required	`approve_members`

Endpoint	Auth	Role check
`GET /api/v1/clubs/:id` pending list	optional Bearer	`pendingMembers` returned only if caller has `manage_club`
`POST /api/v1/sessions/:id/assign`	required	`manage_club` **only when** `session.clubId` is set; if `clubId` is null, only authentication is required

4. Security controls claimed for D02

Implemented & verified

- Network auth is Bearer JWT verification (`requireAuth`); no reliance on unwired `req.auth` session middleware.
- Email register/login uses scrypt hashes (`lib/server/password.ts`) and app HS256 JWTs (`AUTH_JWT_SECRET`).
- Unauthenticated access to protected routes returns `401` (evidence capture).
- Non-manager roster access returns `403` (evidence capture).
- Evidence artifacts redact tokens and passwords.

Not claimed / not verified as D02 deliverables

- Supabase Auth / Supabase JWT as a product integration.
- Production redeploy of this branch.
- Hosted Preview as the primary acceptance environment.

5. Source anchors

- Schema: `prisma/schema.prisma`
- Migrations: `prisma/migrations/**`
- Dispatcher: `lib/server/vercel-dispatch.ts`, `api/v1/[...path].ts`
- Active auth: `lib/server/auth-helpers.ts`, `lib/server/app-jwt.ts`, `app/api/v1/auth/email/**`
- Optional unused helper: `lib/server/supabase-jwt.ts` (see §1)
- Roles: `lib/server/role-checks.ts`
- Deploy adapter: `vercel.json`
- CI: `.github/workflows/backend-verification.yml`
- Local DB setup: `scripts/d02-setup-local-db.sh`